Système et Réseaux TD10 : clients, serveurs et protocoles

avril 2025 — Pierre Rousselin

Comme vu en cours, lorsqu'on lit une *socket* TCP, la lecture est bloquante et retourne :

— soit lorsque la socket distante est fermée, auquel cas la valeur de retour est 0 ;

— soit lorsque *des octets* sont disponibles en lecture.

Mais dans le cas où il y a des octets, il n'y a aucun moyen de savoir à l'avance s'il y aura 1 ou 2 ou ... octets, et s'il faut relancer un `read` ou pas !

Le corollaire de tout ça est qu'en théorie, on ne peut s'arrêter de lire que lorsque, *connaissant le protocole applicatif*, on sait qu'on est arrivé à la fin du message.

Exercice 1 : Stratégie la plus simple : nombre d'octets connus à l'avance

Lorsque le protocole fixe à l'avance le nombre exact d'octets, il suffit de lire *tant que* le nombre total d'octets lus ou écrits n'est pas celui qu'on veut.

Par exemple dans notre INCP (*incrementation protocol*) de base tous les messages font 4 octets.

1. Écrire la fonction `exact_read` spécifiée ci-dessous.

```
/* Lire exactement nbytes ou bien jusqu'à la fin du fichier dans le fichier de
   descripteur fd et les copier à l'adresse buf.
   Retourne le nombre d'octets lu ou bien -1 en cas d'erreur. */
ssize_t exact_read(int fd, void *buf, size_t nbytes);
```

2. Cette stratégie (eventuellement modifiée) est-elle adaptée aux situations suivantes ? Si oui, décrire assez précisément un protocole possible. Si non, dire pourquoi.

— le client envoie un tirage de 10 lettres et le serveur répond l'un des plus longs mots qu'on puisse écrire avec ces lettres ;

— le client et le serveur jouent aux échecs ;

— le client envoie un numéro d'étudiant et le serveur envoie un prénom et un nom.

— le client envoie un nom de fichier et le serveur envoie le contenu du fichier.

--- * ---

Exercice 2 : Un petit protocole de transfert de fichier

1. Sans entrer dans les détails, rappeler les différentes étapes pour qu'un client se connecte avec un serveur en TCP.

2. On suppose que tout se passe bien : le client sait à l'avance quels fichiers sont sur le serveur et tous les fichiers sont dans le même répertoire (le client ne renseigne que le nom de base).

a) Quelle forme peut avoir la requête d'un client dans ce cas ? Comment le serveur sait qu'il peut s'arrêter de lire ?

Faire un choix d'implémentation et écrire les bouts de code correspondants dans le serveur et le client.

b) On suppose ici que le protocole ne prend en charge l'envoi que d'un seul fichier. En quoi est-ce que cela simplifie le problème ? Écrire les bouts de code correspondants :

i) dans le cas où le client écrit le fichier sur sa sortie standard ;

ii) dans le cas où le client stocke le contenu du fichier dans la RAM (utiliser `malloc` et `realloc`).

3. Deux changements : d'abord on veut que le client puisse faire plusieurs requêtes de fichiers avec la même connection. Ensuite, on veut gérer le cas où le fichier :

a) n'est pas présent sur le serveur (fichier non trouvé) ;

b) est présent mais n'a pas le droit d'être partagé (permissions insuffisantes).

La stratégie adoptée va être, pour le serveur, de commencer à envoyer une en-tête (quelques octets) de taille fixe contenant des informations.

Décrire un petit protocole possible pour ces échanges. Écrire les bouts de code correspondant à vos choix.

--- * ---

Exercice 3 : Un exemple de `poll`

On considère le programme suivant :

```
int main(void)
{
        int tube1[2], tube2[2];
        pipe(tube1); pipe(tube2);
        if (fork() == 0) {
                close(tube1[1]); close(tube2[0]);
                dup2(tube1[0], 0); dup2(tube2[1], 1);
                /* line-buffered version of tr a-z A-Z */
                execlp("stdbuf", "stdbuf", "-oL", "tr", "a-z", "A-Z", NULL);
                perror("execlp"); return 1;
        }
        struct pollfd fds[2] =
                { { .fd = 0, .events = POLLIN },
                  { .fd = tube2[0], .events = POLLIN } };
        char buff[4096];
        ssize_t n;
        close(tube1[0]); close(tube2[1]);
        for (;;) {
                if (poll(fds, 2, -1) < 0) {
                        perror("poll"); return 1;
                }
                if (fds[0].revents & POLLIN) {
                        n = read(0, buff, sizeof(buff));
                        write(tube1[1], buff, n);
                }
                if (fds[1].revents & POLLIN) {
                        n = read(tube2[0], buff, sizeof(buff));
                        write(1, buff, n);
                }
        }
        return 0;
}
```

1. Que fait ce programme ? À quoi sert `poll` dans ce programme ?
2. Quel problème de *buffering* est résolu par la commande `stdbuf` chez l'enfant ?
3. Dans `revents`, le bit `POLLHUP` est mis à 1 lorsque le fichier est fermé en écriture. Modifier le programme de façon à ce qu'il se termine lorsque l'utilisateur entre `Ctrl-D` dans le terminal.

--- * ---